

클라우드 인프라 교육 · 3일차 오전

Terraform 기초와 실습

이틀간 클릭으로 만든 것을, 오늘은 코드 한 번으로

이론 40분 — 딱 필요한 것만 (문법 3가지)

실습 2시간 — 6개 실습 + 혼자 해보기 과제

프로젝트 1시간 — 배운 것 총동원 (2개 중 선택)

정리 20분 — destroy와 점검

이론

Terraform

문법 3가지 · 명령 4개 · 기호 3개

1. Terraform이 뭔가요?

"AWS에 이런 것들이 있어야 한다"를 텍스트 파일에 적어 두면,
그대로 만들어 주는 도구

- 이렇게 인프라를 코드로 관리하는 방식을 IaC(Infrastructure as Code)라고 부른다
- 재현성(같은 코드 = 같은 인프라), 협업, 자동화

1. Terraform이 뭔가요?

- **선언형** — 원하는 상태를 적으면 맞춘다
- **멀티 플랫폼** — AWS, GCP, Azure, Docker
- **상태 관리** — 관리 중인 것을 추적
- **미리 보기** — plan 명령어를 통해 설치 전 미리보기

1. Terraform 동작구조

1 당신의 .tf 코드 원하는 인프라를 HCL로 선언

2 Terraform Core 코드·state·실제를 비교해 계획

3 Provider 각 플랫폼 API 호출(AWS...)

4 State 관리 중인 리소스 기록

코드 → Core가 계획 → Provider가 실제 API → State에 기록

1. Terraform 용어

용어	뜻
Provider	플랫폼 API를 다루는 플러그인
Resource	만들 인프라 한 덩어리
Data Source	기존 리소스를 조회만
State	관리 중인 것의 기록(대장)
Module	재사용 가능한 리소스 묶음
Backend	state를 저장하는 위치

1. Terraform 작업폴더 & 파일구조

```
my-project/  
  main.tf          # 리소스 정의  
  variables.tf     # 변수 선언  
  terraform.tfvars # 변수 값  
  outputs.tf       # 출력 정의  
  provider.tf      # provider 설정
```

파일 이름은 자유

폴더 안 모든 .tf를 합쳐 읽음

관례 분리

main/variables/outputs

1. Terraform H C L 문법

```
provider "aws" {           # 연결
  region = "us-east-1"
}

resource "aws_s3_bucket" "data" { # 만들 것
  bucket = "my-bucket"
}
```

provider {}

provider를 어떻게 쓸지

resource {}

실제 인프라 한 덩어리

1. Terraform 데이터타입

```
name    = "web"           # string
count   = 3                # number
enabled = true             # bool
zones   = ["a", "b"]      # list
tags = {                  # map
  env = "dev"
}
```

기본 타입

string · number · bool

집합 타입

list · set · map

구조 타입

object · tuple

2. 문법은 딱 3가지 — 이게 전부입니다 ①

문법 ① 블록: resource "종류" "별명" { ... }

```
resource "aws_s3_bucket" "my_bucket" {  
  ...  
}
```

resource → "자원을 만들겠다"는 선언
"aws_s3_bucket" → 무엇을? (자원 종류 - 정해진 이름)
"my_bucket" → 코드 안에서 부를 별명 (내 마음대로)

읽는 법: "my_bucket이라고 부를 s3 버킷이 있어야 한다"

별명은 AWS 콘솔에 나오지 않는다 — 코드 안에서만 쓰는 이름표

문법 ② 설정: 이름 = 값

```
resource "aws_s3_bucket" "my_bucket" {  
  bucket = "edu-tf-student14-0723"  
  tags   = { Owner = "student14" }  
}  
  
# 블록 { } 안에는 이름 = 값 을 한 줄에 하나씩  
# bucket = ... → AWS에 만들어질 실제 버킷 이름  
# tags   = ... → 붙일 태그  
#  
# 주의: 줄 끝에 쉼표(,) 없음! 세미콜론(;)도 없음!  
#       그냥 줄바꿈이 구분자다
```

어제 콘솔에서 입력칸에 채우던 값들이 = 오른쪽으로 온 것뿐

문법 ③ 참조: 종류.별명.속성

```
resource "aws_vpc" "my_vpc" {  
  cidr_block = "10.0.0.0/16"  
}  
  
resource "aws_subnet" "public" {  
  vpc_id = aws_vpc.my_vpc.id      ← 여기!  
  ...  
}  
  
# "위에서 my_vpc라고 부른 그 VPC의 id를 여기 넣어라"  
# 종류 . 별명 . 속성 - 점(.)으로 연결  
#  
# 어제 콘솔에서 드롭다운으로 "내 VPC 선택"하던 것의 코드 버전
```

이 참조 덕분에 Terraform이 만드는 순서(VPC 먼저 → 서브넷)도 알아서 정한다

문법 3가지 — 한 장 요약

① 블록

```
resource "종류" "별명" { }
```

"이런 자원이 있어야 한다"

② 설정

이름 = 값

"이런 내용으로" (선표 없음!)

③ 참조

종류.별명.속성

"아까 그거 가져와서"

새 문법이 나오면 "이건 ①②③ 중 뭐지?"라고 물어보면 된다 — 답은 항상 셋 중 하나

3. 명령은 4개 — 순서대로 외우세요

```
terraform init      # 준비 (폴더에서 처음 한 번만)
terraform plan      # 미리보기 - "이렇게 하면 뭐가 생기나?"
terraform apply     # 실행 - 진짜로 만들기 (yes 입력)
terraform destroy   # 철거 - 만든 것 전부 삭제 (yes 입력)

# 준비 → 미리보기 → 실행 ... → 철거
#
# 철칙 하나:
#   apply 전에 plan을 읽는다 (견적서 안 보고 결제하지 않기)
```

오늘 이 사이클을 열 번 넘게 돌게 된다 — 손이 외우게 하는 것이 목표

4. plan 미리보기의 기호 3개

기호	뜻	마음가짐
+	새로 만든다	안심
~	있는 것을 고친다	뭔가 바뀌나 확인
-	지운다	정말 지워도 되나 확인!

plan 끝줄 "X to add, Y to change, Z to destroy"의 숫자와 기호만 읽어도 90%는 안전하다

핵심 워크플로우



항상 apply 전에 plan으로 무엇이 바뀔지 확인하는 게 정석.

5. 오늘의 규칙 3개 — 사고 예방

- **규칙 1: 폴더는 하나만** — 오늘 작업은 전부 ~/tf-lab 폴더 안에서. 실습마다 새 폴더 만들지 않기 (디스크가 꽉 차는 사고 방지)
- **규칙 2: ##는 전부 바꾸기** — 코드의 ##를 본인 번호로. 한 군데라도 남으면 오류 (특히 태그 안에 숨은 ## 조심)
- **규칙 3: 확인은 yes** — apply/destroy의 확인 입력은 y가 아니라 yes 전체 타이핑

실습 0

환경 준비

CloudShell 열고 Terraform 깔기 — 딱 4번 복사

② Terraform 설치 (복붙 1·2)

```
# [복붙 1] 다운로드해서 풀기
mkdir -p ~/bin && cd ~/bin && \
curl -sLO https://releases.hashicorp.com/terraform/1.9.8/terraform_1.9.8_linux_amd64.zip && \
unzip -o terraform_1.9.8_linux_amd64.zip && rm terraform_1.9.8_linux_amd64.zip

# [복붙 2] 확인
terraform version
# Terraform v1.9.8 ← 이게 보이면 설치 끝!

# [복붙 3] 오늘의 작업 폴더 만들고 들어가기
mkdir -p ~/tf-lab && cd ~/tf-lab
```

Terraform은 파일 하나짜리 프로그램 — 설치가 이걸로 끝이다

실습 1

첫 자원 — S3 버킷

문법 ①② 그대로 · 명령 4개 첫 완주

① 파일 작성 — main.tf

```
vi main.tf

# --- 아래 내용 입력 (## 두 군데를 본인 번호로!) ---
provider "aws" {
    region = "ap-northeast-2"      # 서울에 만든다
}

resource "aws_s3_bucket" "my_bucket" {
    bucket = "edu-tf-student##-0723" # ← ## 여기 하나
    tags   = { Owner = "student##" }  # ← ## 여기 또 하나!
}

# 방금 쓴 것: 문법① 블록 2개 + 문법② 설정들
# provider 블록 = "어느 클라우드, 어느 리전에" 라는 기본 설정
```

두 군데! 버킷 이름만 바꾸고 태그를 놓치는 실수가 제일 많다

terraform fmt & validate

명령 상세

terraform fmt

- 코드 자동 정렬
- 팀 스타일 통일
- 커밋 전 실행

terraform validate

- 문법·참조 오류 검사
- 인프라 접근 없이 빠름
- plan 전 점검

```
$ terraform fmt      # 자동 정렬
$ terraform validate  # 문법 검증
```

② init → ③ plan — 준비하고 미리보기

```
terraform init
# Terraform has been successfully initialized! ← 성공
# (AWS와 대화하는 부품을 다운로드한 것 - 폴더당 한 번)

terraform plan
#   + resource "aws_s3_bucket" "my_bucket" { ← + 기호!
#       + bucket = "edu-tf-student14-0723"
# Plan: 1 to add, 0 to change, 0 to destroy.

# 읽는 법: "+ 1개 만들 예정, 고치거나 지울 것 없음"
# 아직 아무것도 안 만들어졌다 - plan은 견적서일 뿐
```

plan 끝줄의 숫자 세 개를 소리 내어 읽어 보기 — 이 습관이 사고를 막는다

④ apply — 진짜로 만들기

```
terraform apply
# (견적서가 다시 보인 후)
#   Enter a value: yes      ← yes 전체 타이핑!

# aws_s3_bucket.my_bucket: Creating...
# Apply complete! Resources: 1 added ...

# 확인 1: S3 콘솔 새로고침 → 내 버킷이 생겼다!
# 확인 2: 한 번 더 apply 하면?
terraform apply
# No changes.  ← "이미 원하는 대로니까 안 함"
# 파일 = 원하는 상태, Terraform = 상태를 맞추는 사람
```

클릭 없이 자원이 생기는 첫 순간 — 옆 사람과 버킷 이름 비교해 보기

실습 2

EC2를 코드로

어제의 7단계 클릭 → 블록 2개

① main.tf 맨 아래에 추가

```
vi main.tf      # 기존 내용은 그대로 두고, 맨 아래에 추가

# 최신 아마존 리눅스를 "찾아만 오는" 블록 (만들지는 않음)
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-2023*-x86_64"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.al2023.id    # ← 문법③ 참조!
  instance_type = "t3.micro"
  tags = { Name = "tf-server-student##" } # ← ## 바꾸기
}
```

data = 조회(찾기만), resource = 생성. "찾아온 AMI를 참조해서 서버를 만든다"

② plan → apply → 확인

```
terraform plan
# Plan: 1 to add ...
# (버킷은 이미 있으니 EC2만 + 로 나온다)

terraform apply      # yes
# aws_instance.web: Creating...
# Apply complete!

# EC2 콘솔 확인 → tf-server-student## 실행 중!
#
# 어제 7단계 화면을 오가던 그 일이
# 블록 2개 + 명령 한 번으로 끝났다
```

파일에는 이제 자원이 2개(버킷+서버) — 파일 전체가 "원하는 상태 목록"이다

실습 3

변수 — 반복을 없애기

student##를 딱 한 곳에서만 바꾸게 만들기

① 변수 만들기 — variables.tf (새 파일)

```
vi variables.tf

variable "student_id" {
  description = "본인 아이디 (예: student14)"
  type        = string
}

# variable 블록 = "값을 나중에 넣을 빈 칸을 만든다"
# 왜? 지금 코드엔 student##가 세 군데 -
#   바꿀 때마다 세 군데를 고치는 건 실수의 지름길
#   빈 칸 하나로 모으면 한 곳만 바꾸면 된다
```

같은 폴더의 .tf 파일들은 전부 합쳐져 하나로 읽힌다 — 파일을 나누는 건 정리 목적일 뿐

② 빈 칸 채우기 — terraform.tfvars + 코드 수정

```
vi terraform.tfvars      # 새 파일 - 빈 칸에 넣을 "내 값"

student_id = "student14"    # ← 본인 것으로

# --- 이제 main.tf의 ##들을 변수로 교체 ---
vi main.tf

    bucket = "edu-tf-${var.student_id}-0723"
    tags    = { Owner = var.student_id }
    tags    = { Name = "tf-server-${var.student_id}-v2" }

# var.student_id      → 빈 칸의 값 그대로 쓰기
# "...${var.student_id}..." → 문자열 중간에 끼워넣기

terraform plan    # No changes가 나오면 완벽!
    # (값이 결국 같으니 바꿀 게 없다는 뜻 - 교체 성공의 증거)
```

plan이 "No changes"면 리팩토링 성공 — 모양은 바꿨지만 결과는 같다는 증명

③ 결과 꺼내 보기 — outputs.tf (새 파일)

```
vi outputs.tf

output "server_ip" {
  value = aws_instance.web.public_ip
}

terraform apply      # yes (출력 등록)

# Outputs:
# server_ip = "43.201.xx.xx"

# 콘솔 들어가서 IP 찾던 일이 사라졌다
terraform output server_ip  # 언제든지 다시 보기
```

output = "apply 끝나면 이 값을 화면에 보여줘" — 문법③ 참조를 여기서도 사용

실습 4

어제의 VPC를 코드로

실습 4-B 전체 = 파일 하나 = apply 한 번

시작 전에 — 오늘의 최종 그림

- **만들 것** — VPC + 퍼블릭 서브넷 + IGW + 라우팅 + 보안그룹 + 웹서버 EC2 (어제 손으로 한 그 구성 그대로)
- **파일** — vpc.tf 새 파일 하나에 블록 7개를 차례로 쌓는다 (4번에 나눠 입력)
- **새 문법 없음** — 전부 문법 ①②③의 반복. 블록이 길어질 뿐 원리는 같다
- **완성 판정** — 브라우저에서 내 웹페이지가 뜨면 성공

① vpc.tf (1/4) — VPC · 서브넷 · IGW

```
vi vpc.tf      # 새 파일

resource "aws_vpc" "my_vpc" {
  cidr_block = "10.0.0.0/16"
  tags = { Name = "tf-vpc-${var.student_id}" }
}

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.my_vpc.id      # 참조!
  cidr_block        = "10.0.1.0/24"
  availability_zone = "ap-northeast-2a"
  map_public_ip_on_launch = true            # 서버에 공인 IP 자동 부여
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.my_vpc.id                # 참조!
}
```

어제 순서 그대로: VPC → 서브넷 → IGW. 참조(문법③)가 벌써 두 번

② vpc.tf (2/4) — 라우팅 (인터넷 가는 길)

```
# 이어서 아래에 추가

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.my_vpc.id
  route {
    cidr_block = "0.0.0.0/0"          # 모든 목적지는
    gateway_id = aws_internet_gateway.igw.id # IGW로 (참조!)
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id      # 이 서브넷에
  route_table_id = aws_route_table.public.id # 이 길안내를 연결
}

# 어제의 핵심 문장 기억?
# "IGW로 가는 길이 연결된 서브넷 = 퍼블릭 서브넷"
# 그 연결이 바로 이 association 블록
```

어제 콘솔에서 제일 헷갈렸던 단계가 코드로는 블록 2개 — 다시 보면 개념이 더 선명해진다

③ vpc.tf (3/4) — 보안그룹 (80번 문만 열기)

```
resource "aws_security_group" "web" {
  vpc_id = aws_vpc.my_vpc.id

  ingress {
    # 들어오는 것: 80만 전체 허용
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    # 나가는 것: 전부 허용 (기본 관례)
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

줄마다 이름 = 값 (문법②) — 쉼표 없이 줄바꿈! 어제 규칙표의 코드 버전

④ vpc.tf (4/4) — 웹서버 (초기 설정 포함)

```
resource "aws_instance" "vpc_web" {
  ami           = data.aws_ami.al2023.id
  instance_type = "t3.micro"
  subnet_id     = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y httpd
    systemctl start httpd && systemctl enable httpd
    echo "<h1>Terraform Web - ${var.student_id}</h1>" > /var/www/html/index.html
  EOF

  tags = { Name = "tf-web-${var.student_id}" }
}

output "web_url" {
  value = "http://${aws_instance.vpc_web.public_ip}"
}
```

user_data = 어제 그 시작 스크립트. <<-EOF ~ EOF 사이에 통째로 넣는다

⑤ 실행 — 완성의 순간

```
terraform plan
# Plan: 7 to add ...    ← 블록 7개가 전부 + 로

terraform apply      # yes
# (VPC부터 순서대로 생성되는 로그 - 순서는 참조를 보고
# Terraform이 알아서 정한 것. 우리는 순서를 적은 적 없다!)

# Outputs:
# web_url = "http://43.201.xx.xx"

# 브라우저에서 web_url 열기
# → "Terraform Web - student14" 가 뜨면 완성!!
#
# 어제 30분 = 오늘 2분. 그리고 이 파일은 영원히 재사용 가능
```

성공한 사람은 옆 사람 화면과 IP가 다른지 확인 — 각자 자기 것을 만든 것

실습 5

부분 삭제와 전체 철거

코드에서 지우면 → 현실에서도 지워진다

코드에서 블록을 지우면 어떻게 될까

```
# 실험: main.tf에서 S3 버킷 블록(resource...my_bucket)을  
#       통째로 삭제하고 저장  
  
terraform plan  
#   - resource "aws_s3_bucket" "my_bucket" ...  
# Plan: 0 to add, 0 to change, 1 to destroy.  
  
# 코드에 없다 = 있으면 안 된다 → Terraform이 지우겠다고 함  
terraform apply      # yes → 콘솔에서 버킷 사라짐 확인  
  
# 정리하면:  
#   추가하고 싶다 → 블록을 쓴다  
#   바꾸고 싶다 → 블록을 고친다  
#   지우고 싶다 → 블록을 지운다  
#   전부 밀고 싶다 → terraform destroy
```

파일이 곧 현실 — 이 감각이 잡히면 Terraform은 끝난 것이다. destroy는 프로젝트 후 마지막에!

[트러블슈팅] 오류가 나면 여기부터

```
# 1) "Unclosed configuration block" → 중괄호 { } 짝 안 맞음
#   오류에 나온 줄 번호 근처를 확인

# 2) "Reference to undeclared resource" → 참조 이름 오타
#   문법③의 종류.별명 이 선언과 똑같은지 확인

# 3) "BucketAlreadyExists" → 버킷 이름이 세계 어딘가와 중복
#   본인 번호.날짜 확인

# 4) "InvalidTag" → 태그의 ##를 안 바꿈 (# 는 금지 문자)

# 5) "UnauthorizedOperation" → micro 외 타입 등 정책 위반

# 6) "no space left on device" → 손들기 (강사 호출)
#   보통: rm -rf ~/tf-lab/.terraform 후 다시 init

# 7) 뭐가 꼬였는지 모르겠다 → 일단 plan. plan은 안전하다
```

오류 메시지 첫 줄 + 줄 번호만 읽어도 절반은 해결된다